Ch5 迴圈與生成式

Ch5 迴圈與生成式

- 電腦每秒鐘可執行幾億次的指令，擁有強大的計算能力，程式中迴圈結構可以重複執行某個程式區塊許多次，如此才能善用電腦的計算能力。

- 迴圈結構利用指定迴圈變數的初始條件、迴圈變數的終止條件與迴圈變數的增減值來控制迴圈執行次數。

- 許多問題的解決都涉及迴圈結構的使用
- 例如：
  - 加總、排序、找最大值…等，善用迴圈結構才能有效利用電腦的運算能力與簡化程式碼。

Ch5 迴圈與生成式(ch5\5-for.py)

- 假設要撰寫程式產生1000 個「Hello」，若不使用迴圈結構需寫1000 個「print('Hello')」，如下表。



- 使用迴圈結構可以簡化程式碼達成相同功能，如下表。

產生 1000 個「Hello」的程式碼 ( ：ch5\5-for.py)

```
for i in range(1000):
    print('Hello')
```

Ch5 迴圈與生成式(ch5\5-for.py)

- 函式range 為Python 的內建函式，回傳一個數字串列，使用方式如下表。

使用方法	範例	執行結果
range(終止值) range 函式指定「終止值」，數字串列會到「終止值」的前一個數字爲止，沒有指定起始值，預設起始值爲 0，沒有指定遞增值，預設爲遞增 1。	for i in range(5): print(i)	0 1 2 3 4
range(起始值 , 終止值) range 函式指定「起始值」與「終止值」，數字串列由「起始值」開始到「終止值」的前一個數字爲止，沒有指定遞增值，預設爲遞增 1。	for i in range(2,6): print(i)	2 3 4 5

Ch5 迴圈與生成式(ch5\5-for.py)

使用方法	範例	執行結果
range(起始值 , 終止值 , 遞增 (減) 值) range 函式指定「起始值」、「終止值」與「遞增 (減) 值」，數字串列由「起始值」開始到「終止值」的前一個數字爲止，每次遞增或遞減「遞增 (減) 值」。	for i in range(2,10,2): print(i)	2 4 6 8
	for i in range(100,90,-3): print(i)	100 97 94 91

Ch5 迴圈與生成式

- Python 語言中迴圈結構有for 與while 兩種，迴圈當中可以包含迴圈稱做巢狀迴圈，另外迴圈當中可以跳出迴圈(使用break)，跳過正在執行的迴圈執行迴圈的下一輪(使用continue)。
- 有些迴圈的最後加上else，若迴圈正常結束，不是遇到break 跳出迴圈，就會執行else 程式區塊，以下我們就詳細介紹這些結構。

5-1　迴圈結構 — 使用for

- for 迴圈結構通常用於已知重複次數的程式，迴圈結構中指定迴圈變數的初始值、終止值與遞增(減) 值，迴圈變數將由初始值變化到終止值的前一個數字，每次依照遞增(減) 的值進行數值遞增或遞減。

迴圈結構— 使用for 流程圖

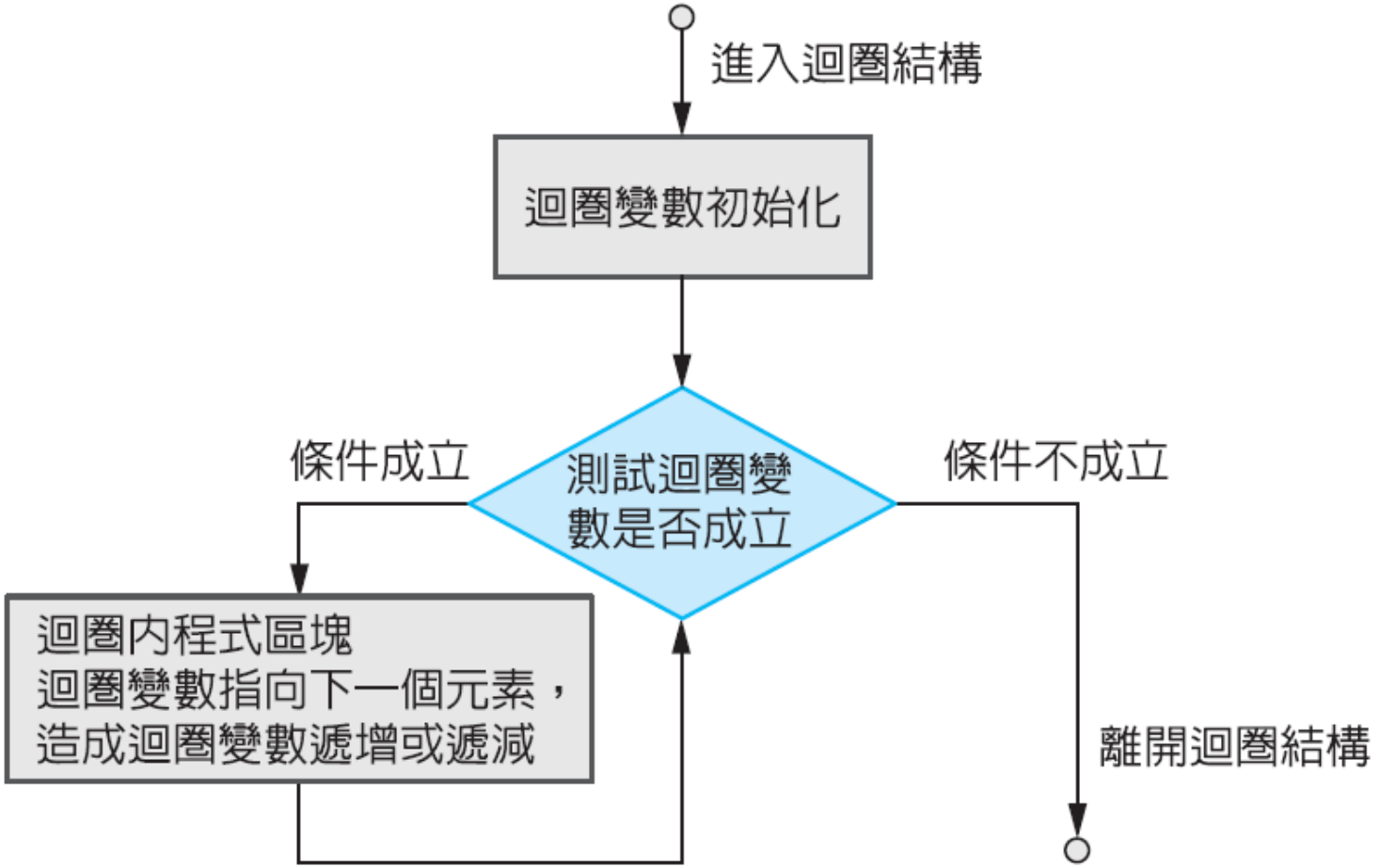


圖 5-1　for 迴圈結構流程圖

表 5-1　for 迴圈結構

for 程式語法	程式範例 (印出 1000 個 Hello)
for 迴圈變數 in range(起始值 , 終止值 , 遞增 (減) 值): 　重覆的程式	for i in range(0, 1000, 1): 　print('Hello')
說明	
for 迴圈內迴圈變數由起始值變化到終止值的前一個數字，每重複執行一次程式迴圈變數就會遞增 (減) 值，重複執行迴圈內程式。	

範例5-1-1 產生ASCII 碼 (ch5\5-1-1-ascii.py)

- 電腦中所有資料皆以二進位方式儲存，大小寫英文字母、數字都有國際標準的二進位編碼，這樣的編碼稱為ASCII 碼

- 例如：
  - A 的ASCII 碼以二進位表示為01000001，十進位表示為65
  - B 的ASCII 碼以二進位表示為01000010，十進位表示為66
  - C 的ASCII 碼以二進位表示為01000011，十進位表示為67，依此類推。
  - 請寫一個程式利用迴圈與「chr」函式，「chr」函式將整數轉換成對應的ASCII 字元。

範例5-1-1 產生ASCII 碼 (ch5\5-1-1-ascii.py)

- 解題想法
  - 可以使用迴圈結構撰寫程式，迴圈變數起始值為輸入的起始值，迴圈變數終止值為輸入的終止值，迴圈每執行一次迴圈變數就會遞增1，迴圈內使用函式chr 將整數轉換成對應的ASCII字元顯示在螢幕上。

範例5-1-1 產生ASCII 碼

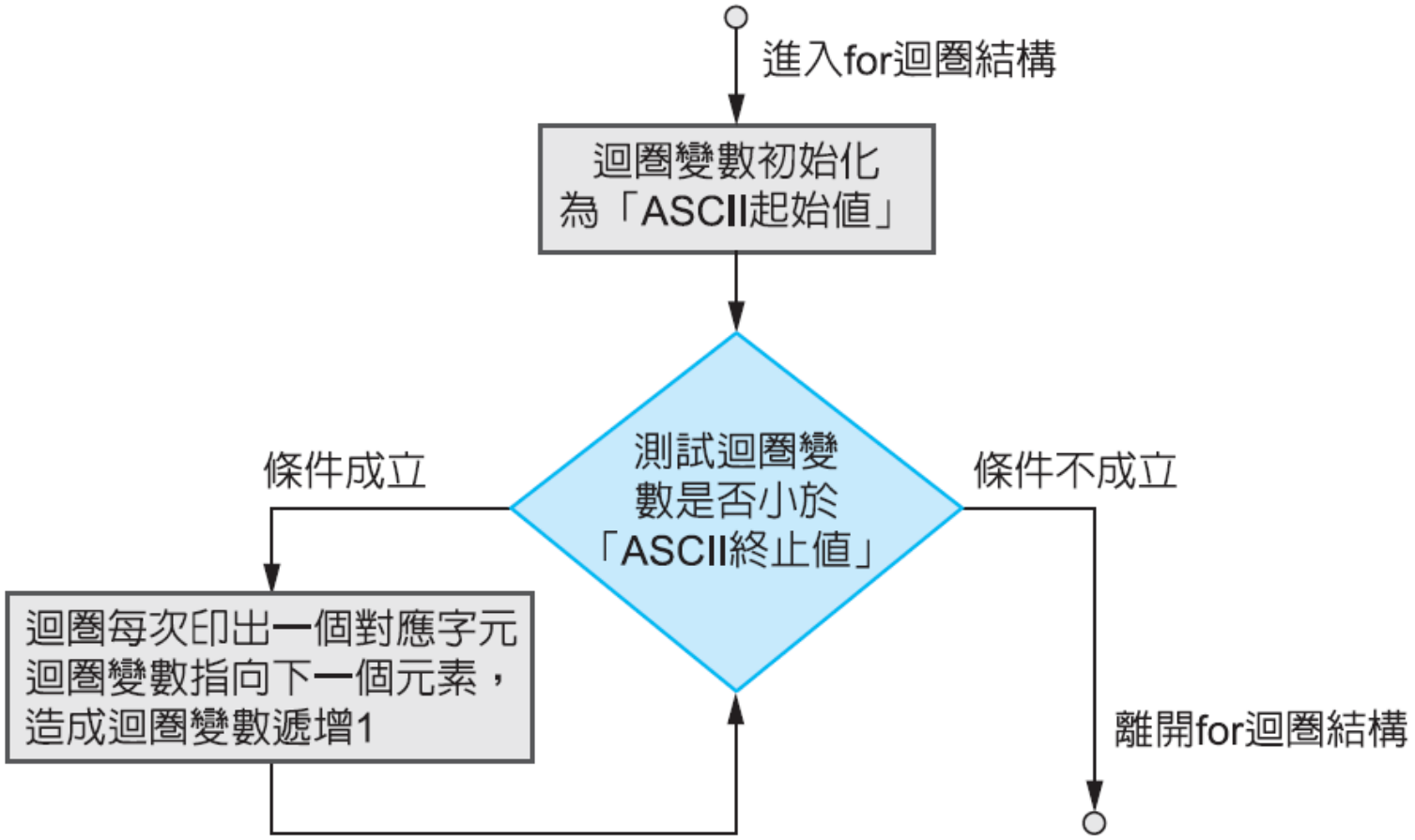


圖 5-2　產生 ASCII 碼流程圖

範例5-1-1 產生ASCII 碼 (ch5\5-1-1-ascii.py)

行數	程式碼	執行結果
1 2 3 4	s = int(input(' 請輸入 ASCII 的起始值？ ')) e = int(input(' 請輸入 ASCII 的終止值？ ')) for i in range(s, e): print(chr(i))	ASCII 起始值輸入「65」，ASCII 終止值輸入「70」，本程式就會顯示 ASCII 介於 65 到 69 的字元。 請輸入 ASCII 的起始值？ 65 請輸入 ASCII 的終止值？ 70 A B C D E

範例5-1-1 產生ASCII 碼 (ch5\5-1-1-ascii.py)

- 程式解說
  - 第1 行：於螢幕輸出「請輸入ASCII 的啟始值？」，經由input 與int 函式，將輸入的值轉換成整數物件，變數s參考到此整數物件。
  - 第2 行：於螢幕輸出「請輸入ASCII 的終止值？」，經由input 與int 函式，將輸入的值轉換成整數物件，變數e參考到此整數物件。
  - 第3 到4 行：使用for 迴圈，其中i 值變化由使用者輸入的ASCII 起始值(s) 到ASCII 終止值(e) 減1，每次遞增1，利用chr 函式，將ASCII 值轉成所對應的字元，並顯示於螢幕。

範例5-1-2 加總 (ch5\5-1-2-sum.py)

- 寫一個程式允許使用者輸入加總的開始值、結束值與遞增值，計算數值加總的結果，例如要計算3+6+9+12 的結果，就輸入3為開始值，13 為結束值，3 為遞增值。

- 解題想法
  - 可以使用迴圈結構撰寫程式，迴圈變數起始值為輸入的加總起始值，迴圈變數終止值為輸入的加總終止值，迴圈每執行一次迴圈變數就會依照輸入的遞增(減) 值進行遞增(減)，迴圈內使用「sum=sum+ 迴圈變數」進行數值的加總，顯示加總的過程。

範例5-1-2 加總

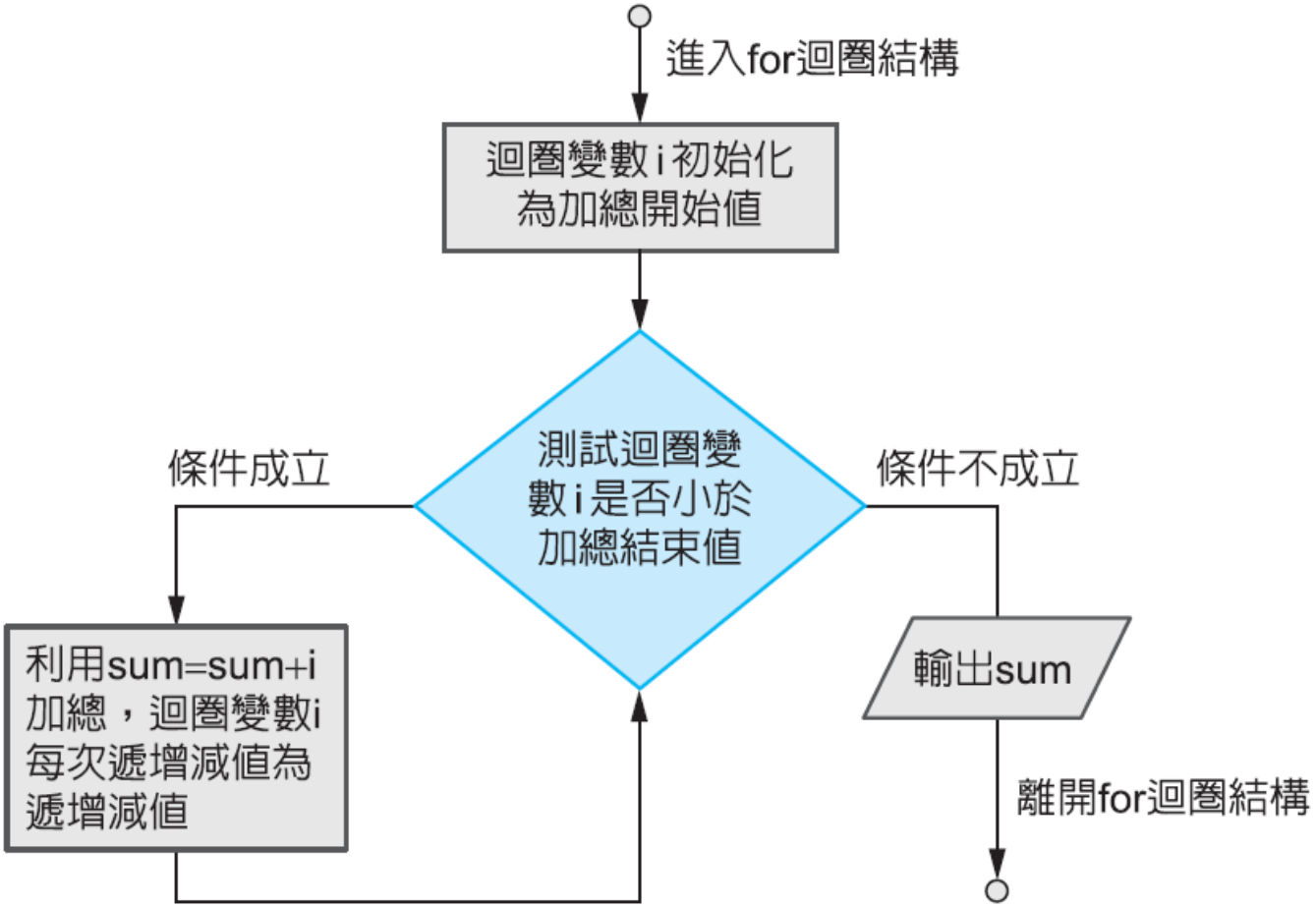


圖 5-3　加總流程圖

範例5-1-2 加總 (ch5\5-1-2-sum.py)

行數	程式碼	執行結果
1 2 3 4 5 6 7	s = int(input(' 請輸入加總開始值？ ')) e = int(input(' 請輸入加總終止值？ ')) inc = int(input(' 請輸入遞增減值？ ')) sum = 0 for i in range(s, e, inc): sum = sum + i print('i 爲 ', i, ' 加總結果爲 ', sum)	加總開始值輸入「3」，加總結束值輸入「13」，加總遞增值輸入「3」，結果如下。 請輸入加總開始值？ 3 請輸入加總終止值？ 13 請輸入遞增減值？ 3 i 爲 3 加總結果爲 3 i 爲 6 加總結果爲 9 i 爲 9 加總結果爲 18 i 爲 12 加總結果爲 30

範例5-1-2 加總 (ch5\5-1-2-sum.py)

- 程式解說
  - 第1 行：於螢幕輸出「請輸入加總開始值？」，經由input 與int 函式，將輸入的值轉換成整數物件，變數s 參考到此整數物件。
  - 第2 行：於螢幕輸出「請輸入加總終止值？」，經由input 與int 函式，將輸入的值轉換成整數物件，變數e 參考到此整數物件。
  - 第3 行：於螢幕輸出「請輸入遞增減值？」，經由input 與int 函式，將輸入的值轉換成整數物件，變數inc 參考到此整數物件。

- 第4 行：初始化變數sum 為0。
- 第5 到6 行：使用for 迴圈，其中i 值變化由使用者輸入的「加總開始值(s)」到「加總終止值(e)」的前一個數字，每次依所輸入的「遞增減值(inc)」進行遞增減，利用「sum=sum+i」計算加總（第6 行），將i 值與sum 值顯示於螢幕（第7 行）。

範例5-1-2 加總 (ch5\5-1-2-sum.py)

- 舉例說明
  - 加總使用sum=sum+i 原理，如下表，在 Python 語言中等號右邊「sum+i」的算式會先計算，結果回存到等號左邊(sum)。

```
sum = 0
for i in range(3, 13, 3):
    sum = sum + i
```

i 值	sum 加總過程	sum 加總後
i=3	sum=0 + 3	sum=3
i=6	sum=3 + 6	sum=9
i=9	sum=9 + 9	sum=18
i=12	sum=18+12	sum=30

範例5-1-2 加總 (ch5\5-1-2-sum2.py)

- 充電時間
  - Python 提供內建的函式sum，會自動將所輸入的所有串列元素加總起來，就不需要寫for迴圈與「sum = sum + i」，修改後的程式如下。

行數	程式碼	執行結果
1 2 3 4	s = int(input(' 請輸入加總開始值？ ')) e = int(input(' 請輸入加總終止值？ ')) inc = int(input(' 請輸入遞增減值？ ')) print(sum(range(s, e, inc)))	加總開始值輸入「3」，加總結束值輸入「13」，加總遞增值輸入「3」，結果如下。 請輸入加總開始值？ 3 請輸入加總終止值？ 13 請輸入遞增減值？ 3 30

5-2 迴圈結構— 使用while

- while 迴圈結構與for 迴圈結構十分類似，while 迴圈結構常用於不固定次數的迴圈，由迴圈中測試條件成立與否，決定是否跳出迴圈，測試條件為真時繼續迴圈，當測試條件為假時結束迴圈。

- 例如：猜數字遊戲
  - 兩人（A 與B）玩猜數字遊戲，一人(A)心中想一個數，另一人(B) 去猜
  - A 就B 所猜數字回答「猜大一點」或「猜小一點」，直到B 猜到A 所想數字，這樣的猜測就屬於不固定次數的迴圈，適合使用while 而不適合使用for。

5-2　迴圈結構— 使用while

- while 指令後面所接測試條件，若為真時會不斷做迴圈內動作，直到測試條件的結果為假時跳出while 迴圈。

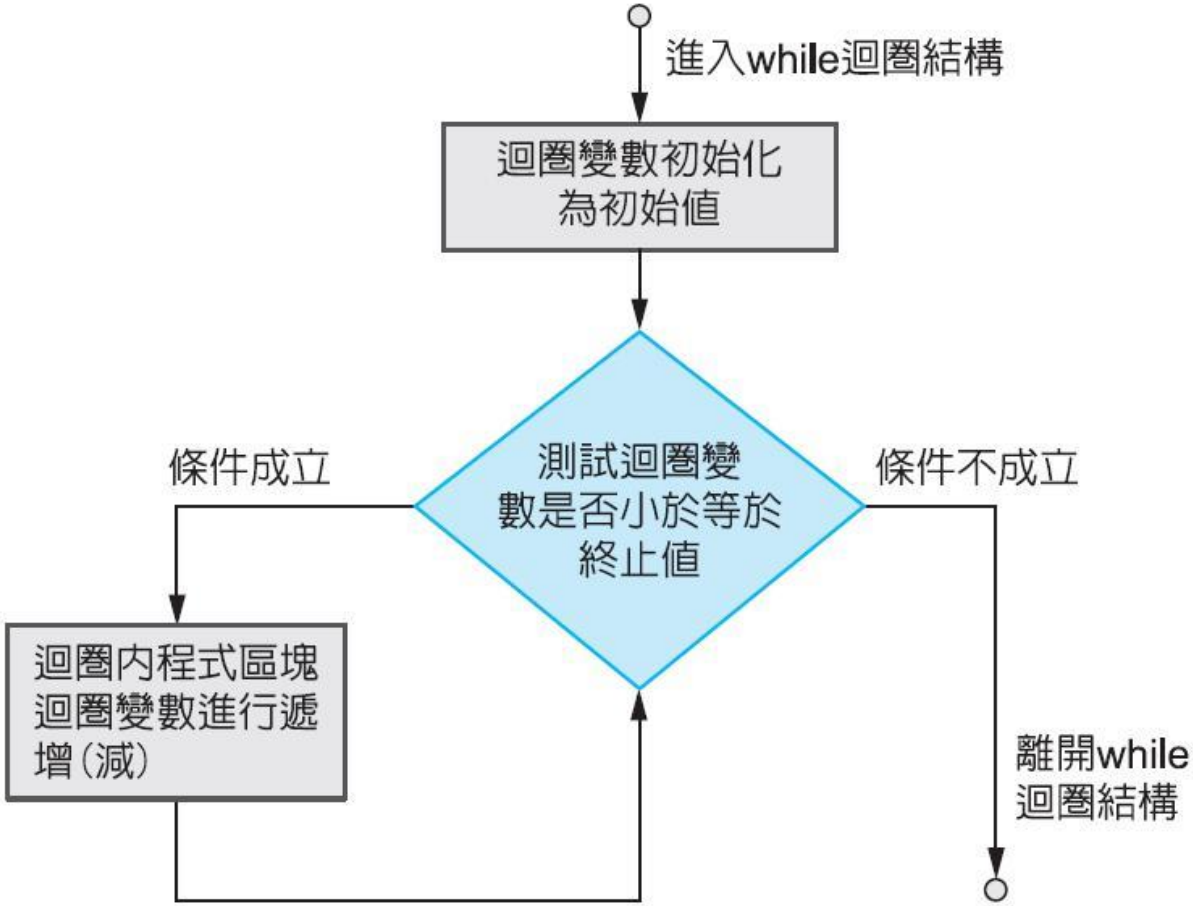


圖 5-4　while 迴圈結構流程圖

表 5-2　while 迴圈結構

while 迴圈語法	程式範例
迴圈變數 = 初始值 while 迴圈變數 <= 終止值 : 　　重覆的程式 　　迴圈變數 = 迴圈變數 + 遞增 (減) 值	i = 3 while i <= 13: 　　sum = sum + i 　　i = i + 3
說明	
while 迴圈內迴圈變數由起始值變化到終止值，每重複執行一次迴圈變數就會遞增 (減) 值，重複執行迴圈內程式，直到超過終止值後停止執行。	

範例5-2-1 階乘計算 (ch5\5-2-1-fac.py)

- 請計算N 為多少時，其階乘值大於等於M。N 階乘表示為N!，其值為「1*2*3*…*(n-1)*n」，使用while 迴圈計算，N! 超過M 的最小N 值為何？

- 解題想法
  - 可以使用迴圈結構撰寫程式，迴圈變數i 起始值為1，進入迴圈之前，測試迴圈變數i的階乘值是否小於M，迴圈每執行一次迴圈變數i 就會遞增1，迴圈內計算迴圈變數i的階乘值，最後顯示「多少階乘會大於等於M」。

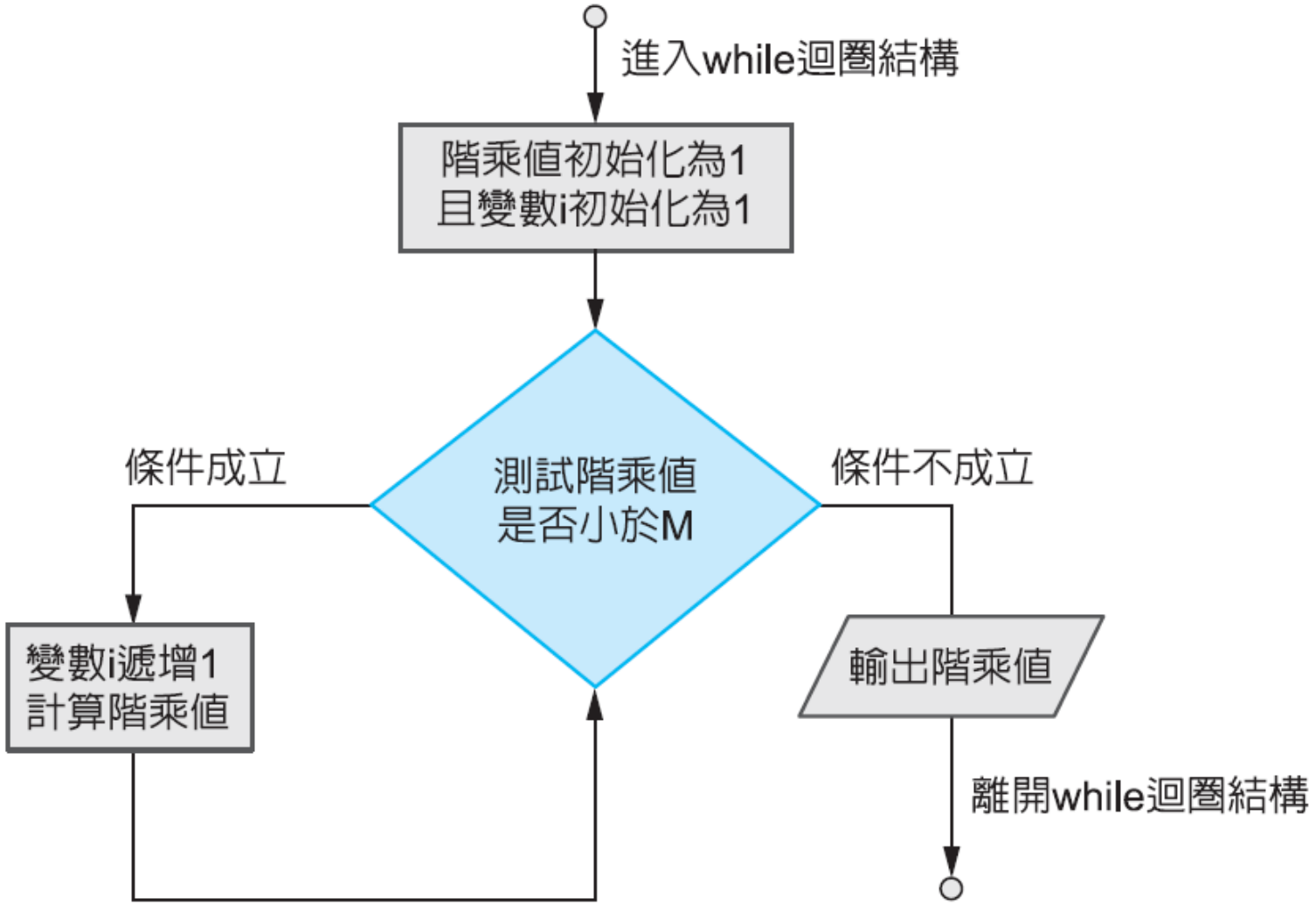

進入while迴圈結構
階乘值初始化為1
且變數i初始化為1
測試階乘值
是否小於M
條件成立
條件不成立
變數i遞增1
計算階乘值
輸出階乘值
離開while迴圈結構

圖 5-5　階乘計算流程圖

範例5-2-1 階乘計算 (ch5\5-2-1-fac.py)

行數	程式碼	執行結果
1	M = int(input(' 請輸入 M ？ '))	輸入 M 值爲「1000」，結果如下。 請輸入 M ？ 1000 7 階乘爲 5040 大於 1000
2	fac = 1	
3	i = 1	
4	while(fac < M):	
5	i = i + 1	
6	fac = fac * i	
7	print(i,' 階乘爲 ', fac,' 大於 ', M)	

範例5-2-1 階乘計算 (ch5\5-2-1-fac.py)

- 程式解說
  - 第1 行：於螢幕輸出「請輸入M ？」，經由函式input 與函式int，將輸入的值轉換成整數物件，變數M 參考到此整數物件。
  - 第2 行：初始化fac 等於1。
  - 第3 行：初始化i 等於1。

- 第4 到6 行：使用while 迴圈，測試fac 是否小於M，i 值每次遞增1，利用「fac=fac*i」求fac，fac 為i 的階乘(第6 行)。
- 第7 行：顯示階乘超過M 的i 值。

範例5-2-1 階乘計算 (ch5\5-2-1-fac.py)

- 舉例說明
  - 「fac = fac * i」的原理，如下表，在Python語言中等號右邊「fac * i」的算式會先計算，結果回存到等號左邊(fac)。

```
M = 1000
fac = 1
i = 1
while(fac < M):
    i = i + 1
    fac = fac * i
print(i,'階乘爲 ', fac,' 大於 ', M)
```

i 值	fac=fac*i 相乘過程	fac 相乘後
i=2	fac= 1*2	fac=2
i=3	fac=2 * 3	fac=6
i=4	fac=6 * 4	fac=24
i=5	fac=24 * 5	fac=120
i=6	fac=120 * 6	fac=720
i=7	fac=720 * 7	fac= 5040

範例5-2-2 猜數字 (ch5\5-2-2-guess.py)

- 大家是否玩過一個遊戲，兩人(A 與B) 一起玩，A 心中想一個數字，B 猜A 心中所想的數字
- B 每猜一次，A 就回答「猜大一點」、「猜小一點」與「猜中了」，當B 猜到A 所想的數字遊戲就結束，我們可以將此遊戲寫成程式，所猜數字介於1 到100。

- 解題想法
  - 利用Python 語言內建模組random 中的隨機函式randint，產生介於1 到99 的目標值
  - 使用while 迴圈結構不斷允許使用者輸入數字進行猜測，測試猜測值與目標值是否相等
  - 若相等則終止迴圈，否則根據猜測值與目標值的大小關係，顯示「猜大一點」、「猜小一點」與「猜中了」等提示。

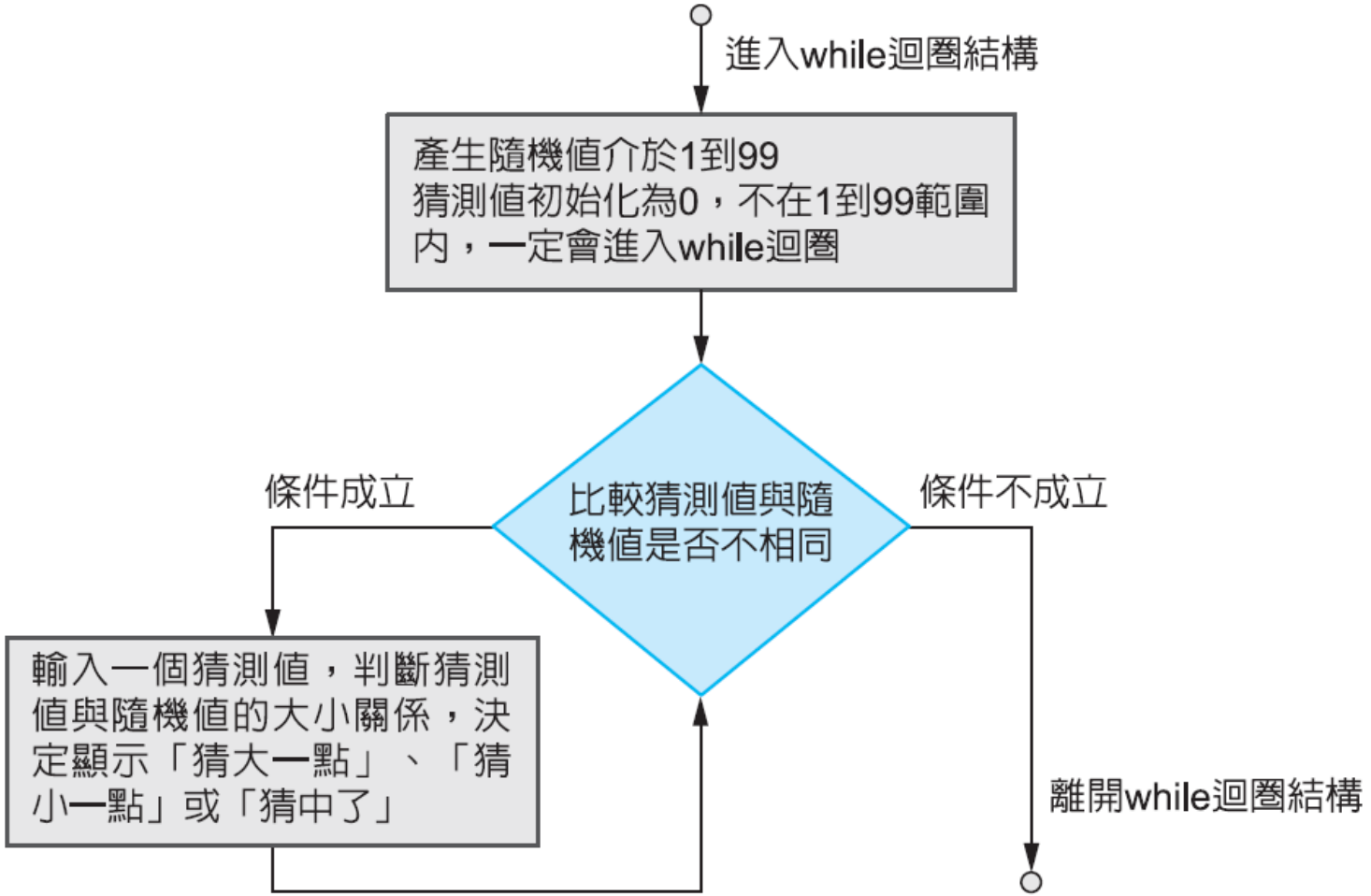

進入while迴圈結構
產生隨機值介於1到99
猜測值初始化為0，不在1到99範圍內，一定會進入while迴圈
比較猜測值與隨機值是否不相同
條件成立
條件不成立
輸入一個猜測值，判斷猜測值與隨機值的大小關係，決定顯示「猜大一點」、「猜小一點」或「猜中了」
離開while迴圈結構

圖 5-6　猜數字流程圖

範例5-2-2 猜數字 (ch5\5-2-2-guess.py)

行數	程式碼	執行結果
1	import random	不斷輸入數字進行猜測，結果如下。
2	target = random.randint(1,99)	
3	guess = 0	請輸入 1 到 99 的數字？ 50
4	while target != guess:	猜大一點
5	guess = int(input(' 請輸入 1 到 99 的數字？ '))	請輸入 1 到 99 的數字？ 75
6	if target < guess:	猜小一點
7	print(' 猜小一點 ')	請輸入 1 到 99 的數字？ 62
8	elif target > guess:	猜小一點
9	print(' 猜大一點 ')	請輸入 1 到 99 的數字？ 57
10	else:	猜中了
11	print(' 猜中了 ')	

範例5-2-2 猜數字 (ch5\5-2-2-guess.py)

- 程式解說
  - 第1 行：包含模組random，為了隨機產生數字。
  - 第2 行：使用模組random 的函式randint 產生1 到99 的隨機數值，指定給變數target。
  - 第3 行：初始化變數guess 為0。
  - 第4 到11 行：執行while 迴圈，當guess 與target 不同時繼續做。

- 第5 行：於螢幕輸出「請輸入1 到99 的數字？」，經由函式input 與函式int，將輸入的值轉換成整數物件，變數guess 參考到此整數物件。
- 第6 到11 行：若target 小於guess，會輸出「猜小一點」（第6 到7 行），否則若target 大於guess，會輸出「猜大一點」（第8 到9 行），否則輸出「猜中了」（第10 到11 行）。

範例5-2-2 猜數字 (ch5\5-2-2-guess.py)

- 充電時間
  - 隨機函式
    - Python 語言包含亂數功能，首先「import random」讓程式有隨機產生數字的功能，接著呼叫random.randint(a, b)，就會產生介於a 到b(包含a 與b) 的隨機整數值。
    - 亂數產生介於1 到99 的數值，公式如下。

```
import random
target = random.randint(1,99)
```

範例5-2-3 複利計算 (ch5\5-2-3-ins.py)

- 請寫一個程式，計算存一筆錢(N) 在銀行，以複利計算，年利率為(X)，計算最少需要幾年才能超過目標金額(M)。

範例5-2-3 複利計算 (ch5\5-2-3-ins.py)

- 解題想法
  - 使用while 迴圈進行複利計算，當還未超過目標金額(M) 時，繼續執行複利計算，當超過目標金額(M) 時，中止while 迴圈。

範例5-2-3 複利計算 (ch5\5-2-3-ins.py)

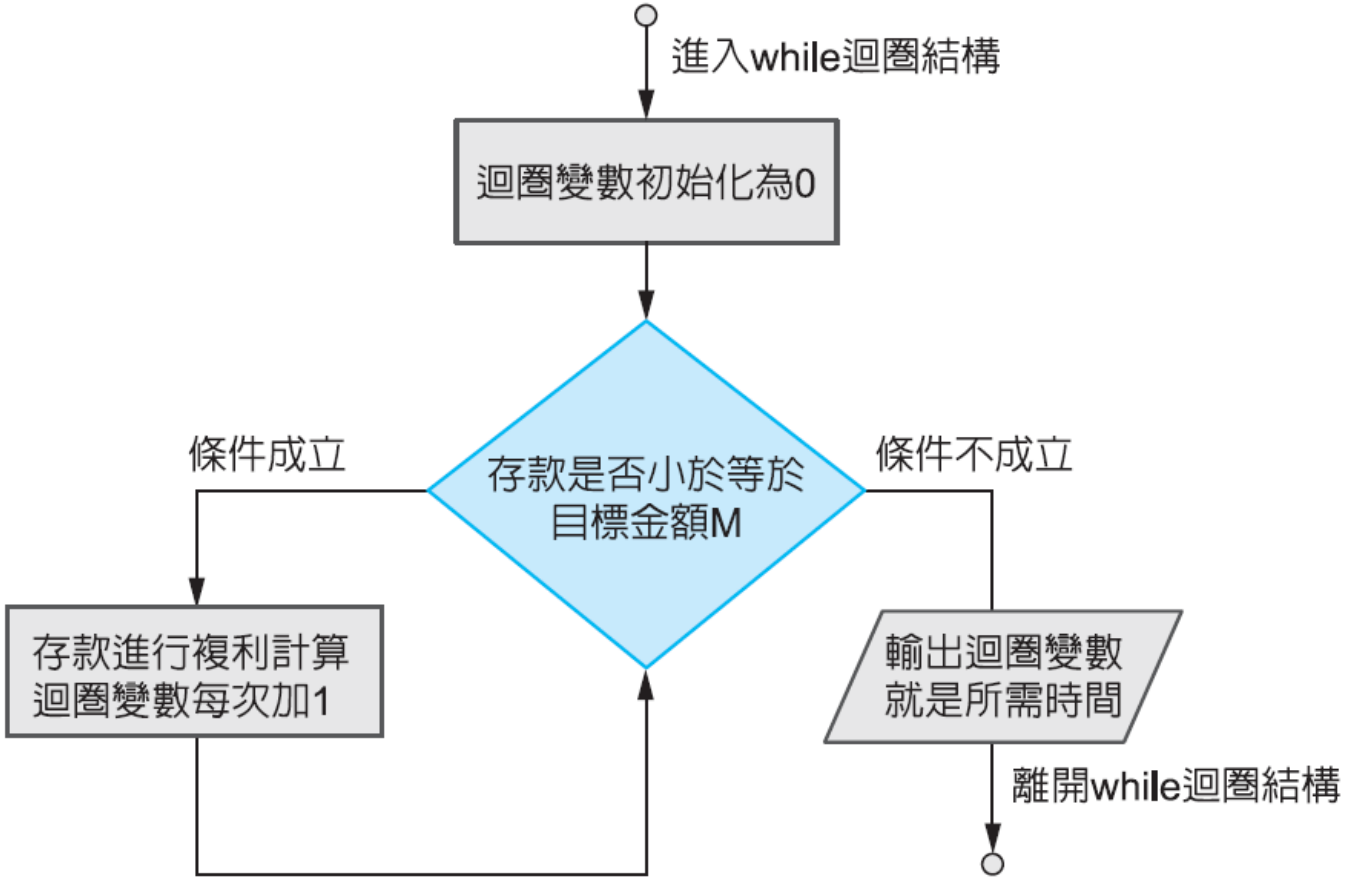


圖 5-7　複利計算流程圖

範例5-2-3 複利計算 (ch5\5-2-3-ins.py)

程式碼

行數	程式碼	執行結果
1	存款金額 = int(input(' 請輸入存款金額？ '))	於存款金額輸入「10000」，存款年利率輸入「10」，目標金額輸入「20000」，結果顯示在螢幕上。 請輸入存款金額？ 10000 請輸入年利率？ 10 請輸入目標金額？ 20000 第 8 年後本利和爲 21435.888100000015 超過 20000
2	年利率 = float(input(' 請輸入年利率？ '))	
3	目標金額 = int(input(' 請輸入目標金額？ '))	
4	i = 0	
5	本利和 = 存款金額	
6	while 本利和 <= 目標金額 :	
7	本利和 = 本利和 * (1 + 年利率 / 100)	
8	i = i + 1	
9	print(" 第 ", i, " 年後本利和爲 ", 本利和 , " 超過 ", 目標金額)	

範例5-2-3 複利計算 (ch5\5-2-3-ins.py)

- 程式解說
  - 第 1 行：使用 input 函式輸入存款金額，使用 int 函式轉成整數物件，變數「存款金額」參考到此整數物件。
  - 第2行：使用input函式輸入年利率，使用float函式轉成浮點數物件，變數「年利率」參考到此浮點數物件。
  - 第 3 行：使用 input 函式輸入目標金額，使用 int 函式轉成整數物件，變數「目標金額」參考到此整數物件。

範例5-2-3 複利計算 (ch5\5-2-3-ins.py)

- 第 4 行：變數 i初始化為 0。
- 第5 行：初始化變數「本利和」為變數「存款金額」。
- 第6行到第9行：使用while迴圈，當本利和小於等於目標金額時，繼續執行迴圈(第6 行)，變數「本利和」為計算本利和的結果(第7 行)，迴圈變數i 每次遞增1(第8 行)。
- 第 9 行：將變數「本利和」超過變數「目標金額」的結果，顯示在螢幕上。

範例5-2-3 複利計算 (ch5\5-2-3-ins.py)

舉例說明

使用公式「本利和 = 本利和 * (1 + 年利率 / 100)」計算本利和的過程，如下表，在 Python 中等號右邊 (本利和 * (1 + 年利率 / 100)) 的算式會先計算，變數「本利和」參考到計算結果。

```
存款金額 = 10000
年利率 = 10
目標金額 = 20000
i = 0
本利和 = 存款金額
while 本利和 <= 目標金額 :
  本利和 = 本利和 * (1 + 年利率 / 100)
  i = i + 1
```

i 值	本利和 * (1 + 年利率 / 100) 相乘過程	本利和 * (1 + 年利率 / 100) 相乘後
i=1	N=10000*(1+10/100)	N=11000
i=2	N=11000*(1+10/100)	N=12100
i=3	N=12100*(1+10/100)	N=13310
i=4	N=13310*(1+10/100)	N=14641
i=5	N=14641*(1+10/100)	N=16105.1
i=6	N=16105.1*(1+10/100)	N=17715.61
i=7	N=17715.61*(1+10/100)	N=19487.171
I=8	N=19487.171*(1+10/100)	N=21435.8881

範例5-2-4 計算密碼 (ch5\5-2-4-password.py)

- 小華回家時發現，大門換上了新的密碼鎖，並且收到爸媽的簡訊通知，密碼為從1 開始累加所有奇數(1、3、5、7、9、…) 除以1009 的餘數，等於自己的生日，符合的第一個累加值就是密碼
- 小華生日為9 月11 日(轉換成911)，小華拿出紙筆開始計算，發現需要花很多時間且計算容易出錯，請寫出一個程式幫助小華計算出密碼，流程圖表示如下。

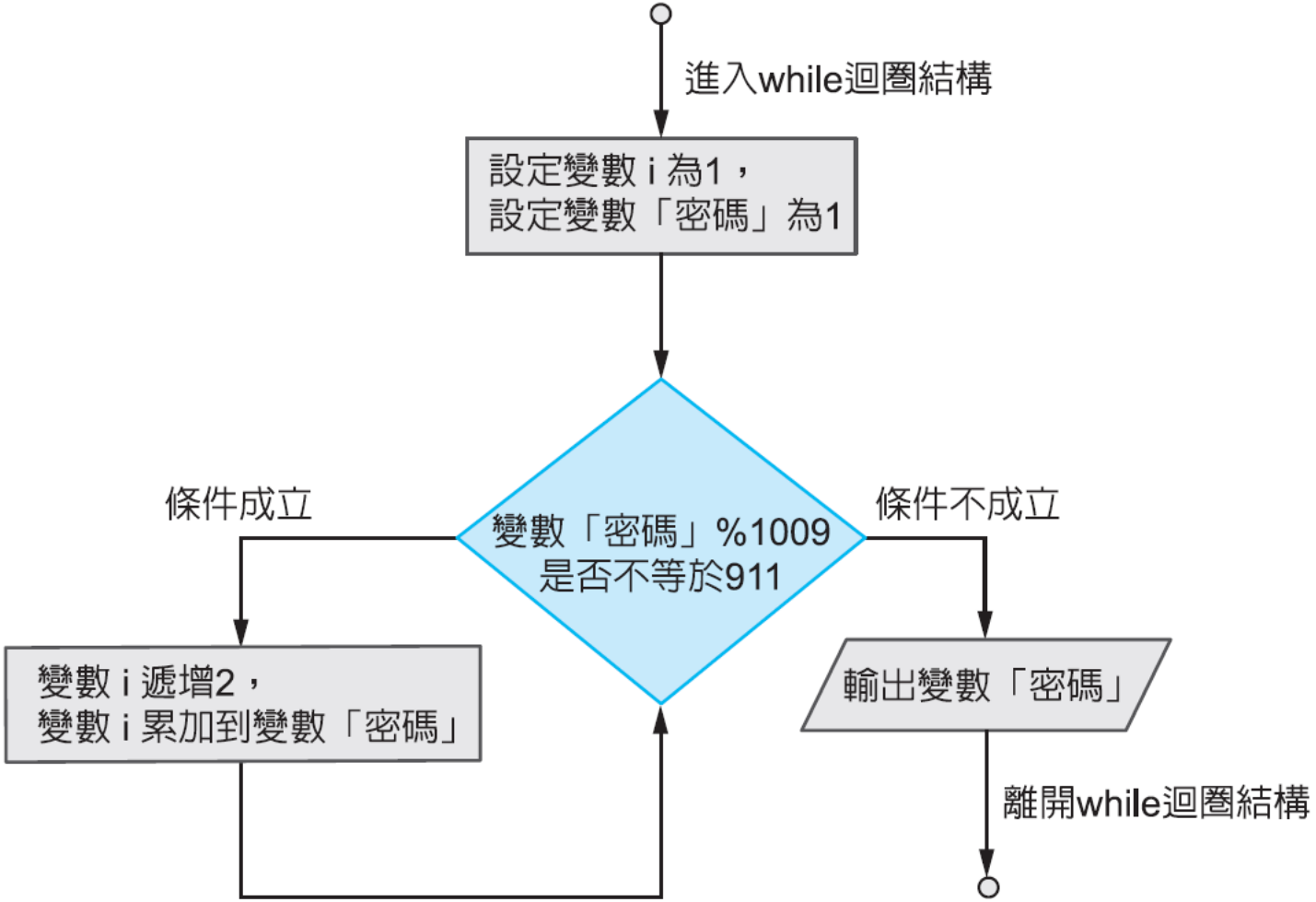

進入while迴圈結構
設定變數 i 為1，
設定變數「密碼」為1
變數「密碼」%1009
是否不等於911
條件成立
條件不成立
變數 i 遞增2，
變數 i 累加到變數「密碼」
輸出變數「密碼」
離開while迴圈結構

圖 5-8　計算密碼流程圖

程式碼

- i=1
- 密碼=1
- while 密碼%1009 != 911:
- 　　i = i + 2
- 　　密碼 = 密碼 + i
- print(密碼)

執行結果

大門密碼如下。

```
148225
```

程式解說

- **第1 行**：設定變數i 為1。
- **第2 行**：設定變數「密碼」為1。
- **第3 行到第5 行**：使用while 迴圈，當密碼除1009 的餘數不等於911 時執行while迴圈，變數i 遞增2 (第4 行)，變數「密碼」累加變數i(第5 行)。
- **第6 行**：顯示變數「密碼」到螢幕上。

5-3 巢狀迴圈

- 巢狀迴圈並不是新的程式結構，只是迴圈範圍內又有迴圈，巢狀迴圈可以有好幾層，巢狀迴圈與單層迴圈運作原理相同，從外層迴圈來看，內層迴圈只是外層迴圈內的動作，因此外層迴圈作用一次，內層迴圈需要執行完畢。
- 以輸出九九乘法表為例，當外層迴圈作用一次，內層迴圈要執行九次，當外層迴圈作用九次，內層迴圈總共執行八十一次。

範例5-3-1 九九乘法表 (ch5\5-3-1-99.py)

- 寫一個程式印出九九乘法表。
- 解題想法
  - 巢狀迴圈的外層迴圈使用迴圈變數i，內層迴圈使用迴圈變數j，外層迴圈i 等於1，內層迴圈j 由1 變化到9，印出「1*1=1，1*2=2，1*3=3，…，1*9=9」，
  - i 遞增1，外層迴圈i 等於2，內層迴圈j 由1 變化到9，印出「2*1=2，2*2=4，2*3=6，…，2*9=18」，
  - 依此類推，直到外層迴圈i 等於9，內層迴圈j 由1 變化到9，印出「9*1=9，9*2=18，9*3=27，…，9*9=81」。

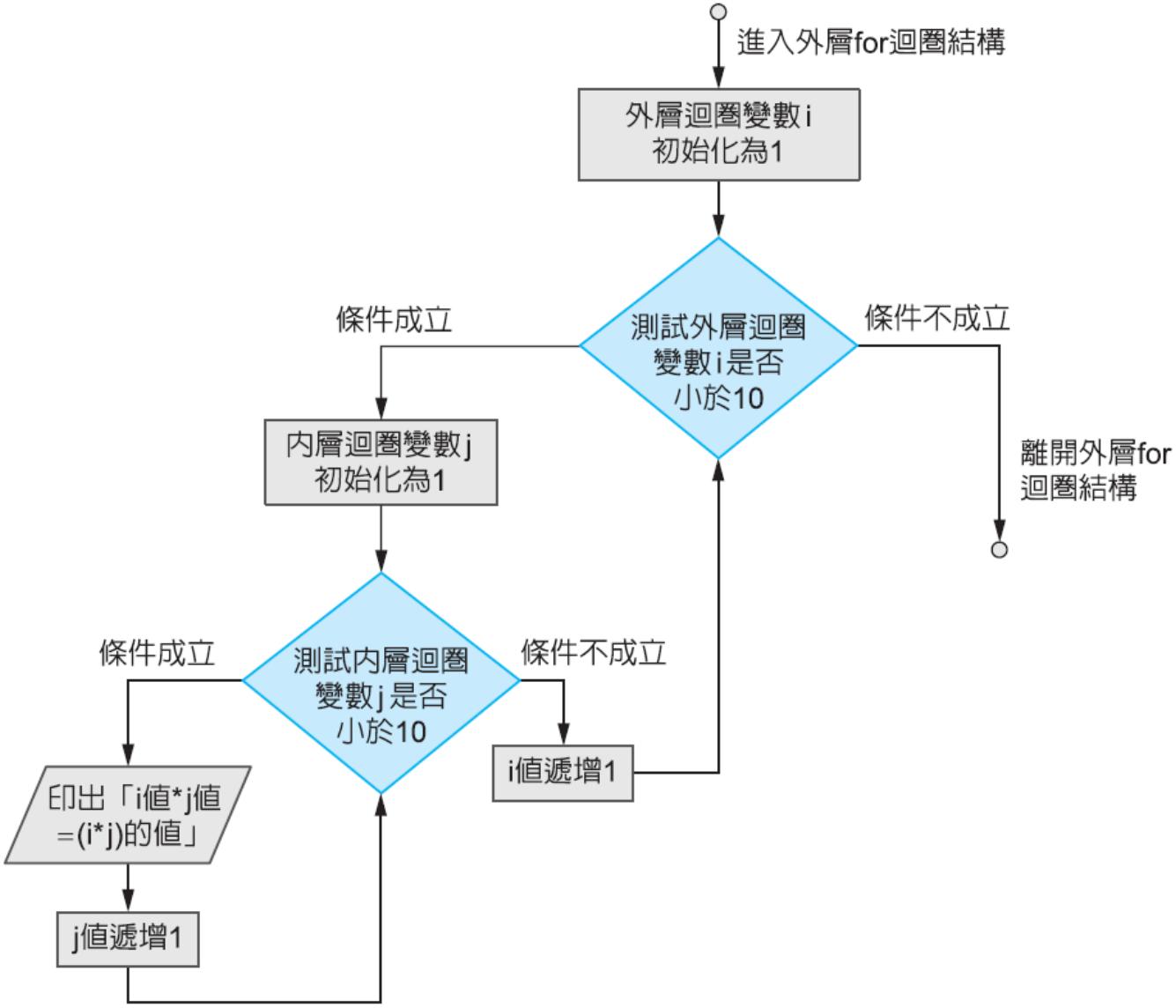


圖 5-9　九九乘法表流程圖

範例5-3-1 九九乘法表 (ch5\5-3-1-99.py)

```
1  for i in range(1,10):
2      for j in range(1,10):
3          print(i, '*', j, '=', i*j, '\t', sep='',end='')
4      print()
```

執行結果

```
1*1=1   1*2=2   1*3=3   1*4=4   1*5=5   1*6=6   1*7=7   1*8=8   1*9=9
2*1=2   2*2=4   2*3=6   2*4=8   2*5=10  2*6=12  2*7=14  2*8=16  2*9=18
3*1=3   3*2=6   3*3=9   3*4=12  3*5=15  3*6=18  3*7=21  3*8=24  3*9=27
4*1=4   4*2=8   4*3=12  4*4=16  4*5=20  4*6=24  4*7=28  4*8=32  4*9=36
5*1=5   5*2=10  5*3=15  5*4=20  5*5=25  5*6=30  5*7=35  5*8=40  5*9=45
6*1=6   6*2=12  6*3=18  6*4=24  6*5=30  6*6=36  6*7=42  6*8=48  6*9=54
7*1=7   7*2=14  7*3=21  7*4=28  7*5=35  7*6=42  7*7=49  7*8=56  7*9=63
8*1=8   8*2=16  8*3=24  8*4=32  8*5=40  8*6=48  8*7=56  8*8=64  8*9=72
9*1=9   9*2=18  9*3=27  9*4=36  9*5=45  9*6=54  9*7=63  9*8=72  9*9=81
```

範例5-3-1 九九乘法表 (ch5\5-3-1-99.py)

- 程式解說
  - 第1 到4 行：變數i 為外層迴圈，其變化值由1到9，其範圍包含內層迴圈。

- 程式解說
  - 第2 到3 行：變數j 為內層迴圈，其變化值由1到9，當外層迴圈執行一次，內層迴圈執行九次
  - 依序將「變數i* 變數j= 變數i 與j 相乘結果」顯示在螢幕（第3 行）
  - 接著串接tab 字元「'\t'」，且設定sep 為空字串「''」取代原本的空白字元，與設定end 為空字串「''」取代原本的換行字元「'\n'」。

- 程式解說
  - 第4行：print() 表示換行，內層迴圈執行完畢後進行換行。

範例5-3-1 九九乘法表 (ch5\5-3-1-99.py)

- 執行結果說明
  - 巢狀迴圈印出九九乘法表原理，如下表，在 Python 語言中外層迴圈包含內層迴圈，外層迴圈執行一次，內層迴圈要執行完畢，九九乘法表外層迴圈執行九次，內層迴圈執行八十一次。

```
for i in range(1,10):
    for j in range(1,10):
        print(i, '*', j, '=', i*j, ' ', sep='',end='')
    print()
```

i 值	j 值	輸出結果
i=1	j=1,2,3,4,5,6,7,8,9	1*1=1 1*2=2 1*3=3 1*4=4 1*5=5 1*6=6 1*7=7 1*8=8 1*9=9
i=2	j=1,2,3,4,5,6,7,8,9	2*1=2 2*2=4 2*3=6 2*4=8 2*5=10 2*6=12 2*7=14 2*8=16 2*9=18
i=3	j=1,2,3,4,5,6,7,8,9	3*1=3 3*2=6 3*3=9 3*4=12 3*5=15 3*6=18 3*7=21 3*8=24 3*9=27
…	…	…
i=9	j=1,2,3,4,5,6,7,8,9	9*1=9 9*2=18 9*3=27 9*4=36 9*5=45 9*6=54 9*7=63 9*8=72 9*9=81

範例5-3-2 印星號 (ch5\5-3-2-star.py)

- 請使用巢狀迴圈印出如右圖所示的星號，第一行一個星號，第二行兩個星號，…，第五行五個星號，全部靠左排列。

```
*
**
***
****
*****
```

範例5-3-2 印星號(ch5\5-3-2-star.py)

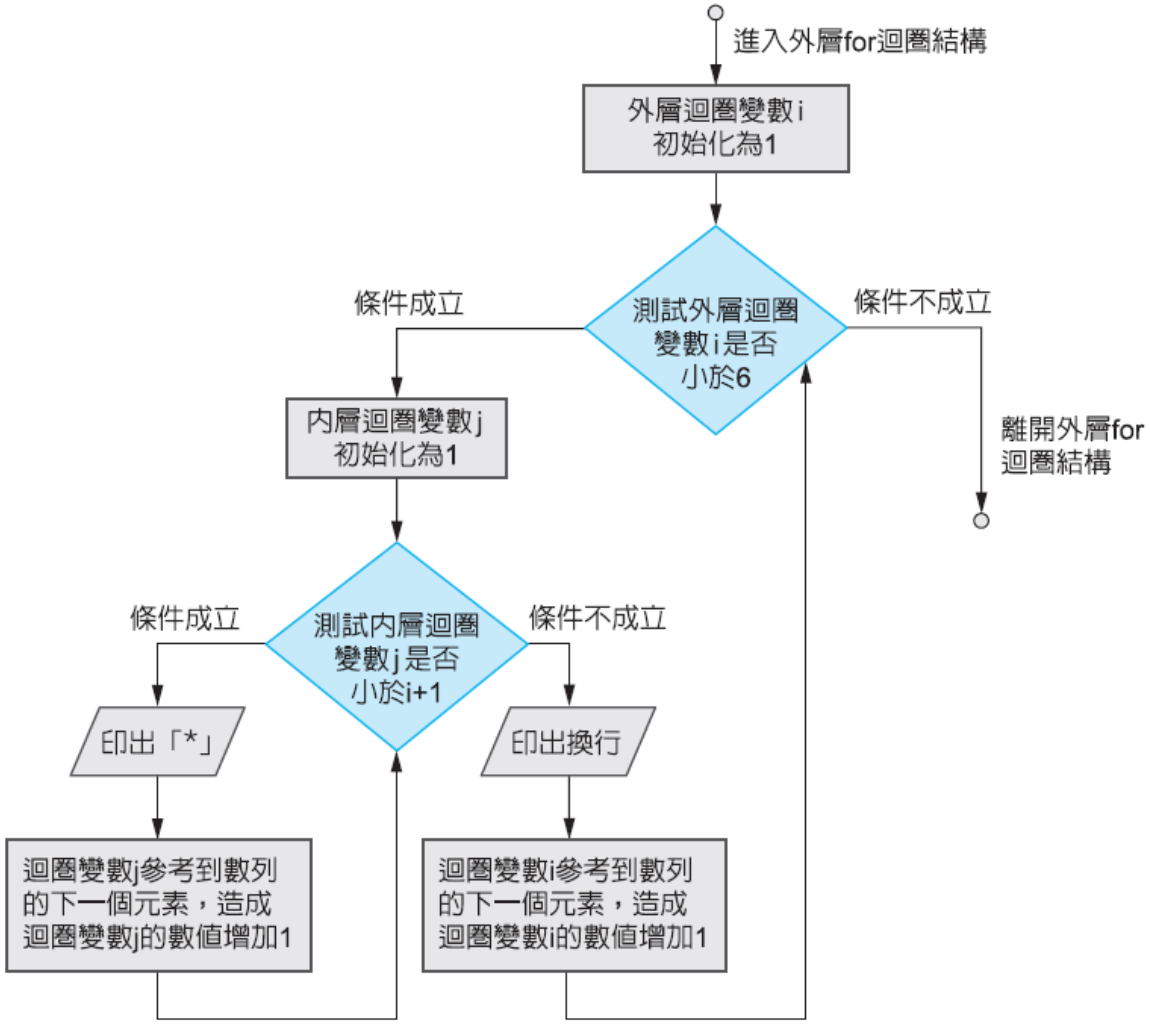


圖 5-10 印星號流程圖

範例5-3-2 印星號 (ch5\5-3-2-star.py)

程式碼

行數	程式碼	執行結果
1	for i in range(1,6):	*
2	for j in range(1,i+1):	**
3	print("*", end="")	***
4	print()	****
		*****

範例5-3-2 印星號 (ch5\5-3-2-star.py)

- 程式解說
  - 第 1 到 4 行：使用巢狀 for 迴圈，外層迴圈變數 i 由 1 到 5，迴圈變數 i 參考到數列的下一個元素，造成迴圈變數i 的數值增加1 (第1 行)。內層迴圈變數j 由1 到i，迴圈變數j 參考到數列的下一個元素，造成迴圈變數j 的數值增加1 (第2 行)。
  - 內層迴圈內使用函式print 印出星號(*)，接著設定end 為空字串，表示不自動換行(第3 行)。
  - 外層迴圈內使用函式print 進行換行，表示內層迴圈執行結束後加上換行，函式print 內參數end 預設為換行字元(\n)，所以「print()」可以達成換行效果。

範例5-3-2 印星號 (ch5\5-3-2-star.py)

舉例說明

巢狀迴圈印出星號原理，如下表。外層迴圈包含內層迴圈，外層迴圈執行一次，內層迴圈要執行完畢。本範例內層迴圈執行次數受外層迴圈限制，i 等於 1，j 由 1 依序變化到 1；i 等於 2，j 由 1 依序變化到 2；i 等於 3，j 由 1 依序變化到 3；i 等於 4，j 由 1 依序變化到 4；i 等於 5，j 由 1 依序變化到 5。

```
for i in range(1,6):
    for j in range(1,i+1):
        print("*", end="")
    print()
```

i 值	j 值	輸出結果
i=1	j=1	*
i=2	j=1,2	**
i=3	j=1,2,3	***
i=4	j=1,2,3,4	****
i=5	j=1,2,3,4,5	*****

範例5-3-2 印星號 (ch5\5-3-2-star.py)



充電時間

若要印出以下結果，請問該如何印出？

```
*
**
***
****
***
**
*
```

1 先印上半部

2 本範例外層迴圈的迴圈變數改成由大到小，由 3 到 1 每次遞減 1，使用「-1」表示遞減 1。

提示

可以切割成兩部分，先印上半部，再使用一次巢狀迴圈印出下半部，如圖所示。

5-4 迴圈結構特殊指令的使用—break、continue 與else

- 迴圈在特殊需求下可以使用break、continue 與else 指令。
  - 當要跳出迴圈時可以使用break 跳出迴圈。
  - 當要跳過迴圈內之後的程式碼，迴圈變數值直接遞增或遞減，繼續迴圈的執行，則使用continue，也就是跳過continue 後的程式繼續執行迴圈程式。
  - 若迴圈正常結束，就會執行else 程式區塊，若迴圈經由break 中斷，就不會執行else程式區塊。

5-4 迴圈結構特殊指令的使用—break、continue 與else

針對不同的迴圈結構執行 break，如下表。( ：ch5\5-4-break.py)

表 5-3 針對不同的迴圈結構執行 break

結構	範例	執行結果	說明

for	for i in range(1,6):
	  print(i)
	  if i == 3:
	  break
		1
		2
		3
			當 i 等於 3 時，中斷 for 迴圈，所以只印出「1、2、3」。

while	i = 1
	while (i<6):
	  print(i)
	  if i == 3:
	  break
	  i += 1
		1
		2
		3
			當 i 等於 3 時，中斷 while 迴圈，所以只印出「1、2、3」。

5-4 迴圈結構特殊指令的使用—break、continue 與else

針對不同的迴圈結構執行 continue，如下表。(：ch5\5-4-continue.py)

表 5-4 針對不同的迴圈結構執行 continue

結構	範例	執行結果	說明

for	for i in range(1,6):
	  if i == 3:
	  continue
	  print(i)
		1
		2
		4
		5
			當 i 等於 3 時，跳到 for 迴圈的開頭繼續執行，所以印出「1、2、4、5」。

結構	範例	執行結果	說明

while	i = 0
	while (i<5):
	  i += 1
	  if i == 3:
	  continue
	  print(i)
		1
		2
		4
		5
			當 i 等於 3 時，跳到 while 迴圈的開頭繼續執行，且 i 值加 1，所以印出「1、2、4、5」。

5-4 迴圈結構特殊指令的使用—break、continue 與else

針對不同的迴圈結構執行 else，如下表。( ：ch5\5-4-else.py)

表 5-5 針對不同的迴圈結構執行 else

結構	範例	執行結果	說明

for	for i in range(1,6):
	  print(i)
	  if i == 6:
	  break
	else:
	  print('for 迴圈正常結束 ')
		1
		2
		3
		4
		5
		for 迴圈正常結束
			當 i 等於 6 時，才跳出 for 迴圈，所以 for 迴圈正確執行完畢，沒有執行 break，所以印出「1、2、3、4、5」與「for 迴圈正常結束」。

while	i = 1
	while (i<6):
	  print(i)
	  if i == 6:
	  break
	  i += 1
	else:
	  print('while 迴圈正常結束 ')
		1
		2
		3
		4
		5
		while 迴圈正常結束
			當 i 等於 6 時，才跳出 while 迴圈，所以 while 迴圈正確執行完畢，沒有執行 break，所以印出「1、2、3、4、5」 與「while 迴圈正常結束」。

5-4 迴圈結構特殊指令的使用—break、continue 與else

- 將break、continue 與else 可以使用在迴圈中，產生想要的結果
- 例如：使用帳號與密碼登入系統，若帳號密碼正確，則使用break 中斷迴圈進入系統，否則繼續停留在登入畫面，使用帳號與密碼登入系統。

範例5-4-1 登入系統 (ch5\5-4-1-login.py)

- 請寫一個程式模擬帳號與密碼登入，使用者輸入帳號與密碼，若帳號密碼一致，則輸出「帳號與密碼正確」，否則輸出「登入失敗」。

- 解題想法
  - 使用while 迴圈，迴圈內允許使用者輸入帳號與密碼，若帳號與密碼正確，則顯示「帳號與密碼正確」，接著使用break 中斷while 迴圈，否則顯示「登入失敗」。
  - 使用「while True:」無窮迴圈結構，表示永遠測試條件都成立，允許使用者不斷輸入帳號與密碼直到執行break 才中斷「while True:」無窮迴圈。

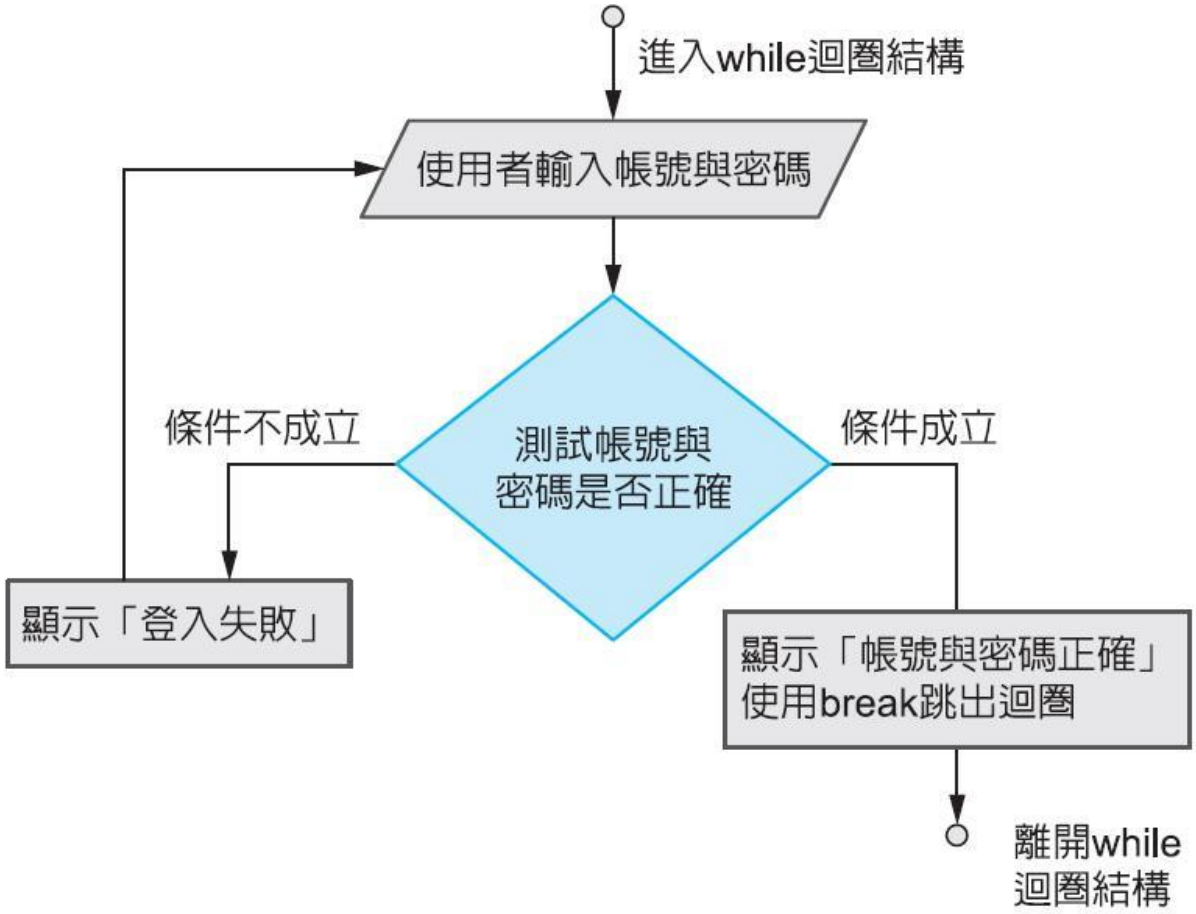


圖 5-11　登入系統流程圖

範例5-4-1 登入系統 (ch5\5-4-1-login.py)

行數	程式碼	執行結果
1	while True:	預設登入帳號爲「abc」，密碼爲「123」，帳號輸入「abc」與密碼輸入「12」，顯示「登入失敗」，接著帳號輸入「abc」與密碼輸入「123」，顯示「帳號與密碼正確」。
2	acc = input(' 請輸入帳號？ ')	
3	pwd = input(' 請輸入密碼？ ')	
4	if (acc == 'abc' and pwd == '123'):	
5	print(' 帳號與密碼正確 ')	
6	break	
7	else:	
8	print(' 登入失敗 ')	

```
請輸入帳號？ abc
請輸入密碼？ 12
登入失敗
請輸入帳號？ abc
請輸入密碼？ 123
帳號與密碼正確
```

範例5-4-1 登入系統 (ch5\5-4-1-login.py)

- 程式解說
  - 第1 行到第8 行：while 迴圈中測試條件為True，表示測試永遠為真，表示無窮迴圈(第1 行)，登入成功後使用break 跳出迴圈，中斷無窮迴圈執行(第6 行)。
  - 第2 行：於螢幕輸出「請輸入帳號？」，經由函式input輸入帳號，變數 acc 參考到輸入的帳號。

- 第3 行：於螢幕輸出「請輸入密碼？」，經由函式input輸入密碼，變數 pwd 參考到輸入的密碼。
- 第4 行到第8 行：判斷帳號是否等於帳號「abc」，密碼是否等於密碼「123」(第4行)
  - 若是則顯示「帳號與密碼正確」訊息(第5 行)
  - 使用break 跳出迴圈(第6 行)
  - 否則顯示「登入失敗」訊息，繼續while 迴圈(第7 行到第8 行)。

5-5 for 迴圈與資料儲存容器

- 若要取出資料儲存容器(tuple、串列、字典與集合) 的所有元素，可以使用「for」一個一個取出每一個元素，再對每一個元素進行計算。

範例 5-5-1 使用 for 讀取 tuple

( ：ch5\5-5-1-for-tuple.py)

範例	執行結果	說明

t = tuple('Tuple')
for i in t:
  print(i)
for i in range(0,len(t)):
  print(t[i])
	T
	u
	p
	l
	e
	T
	u
	p
	l
	e
		※t 爲 tuple 物件，內容爲「'T', 'u', 'p', 'l', 'e'」。
		※ 使用「for i in t:」，讀取 t 中每個元素指定給變數 i。
		※ 使用「for i in range(0,len(t)):」，i 爲 0 到 t 的長度減 1，利用 t[i] 讀取 t 的每一個元素。

5-5 for 迴圈與資料儲存容器

範例 5-5-2 使用 for 讀取串列

(: ch5\5-5-2-for-list.py)

範例	執行結果	說明

sh = ['牛奶', '蛋', '咖啡豆']
for i in range(0,len(sh)):
  print (i, sh[i])
for i, name in enumerate(sh, start=1):
  print(i, name)
	0 牛奶
	1 蛋
	2 咖啡豆
	1 牛奶
	2 蛋
	3 咖啡豆
		※sh 爲串列，內容爲「'牛奶', '蛋', '咖啡豆'」。
		※ 使用「for i in range(0,len(sh)):」，i 爲 0 到 sh 的長度減 1，利用 sh[i] 取 sh 的每一個元素。
		※ 使用「for i, name in enumerate(sh, start=1):」，函式 enumerate 會將 sh 每個元素編號，預設由 0 開始編號，因爲「start=1」，所以修改成由 1 開始編號，會回傳編號與元素內容，編號指定給變數 i，元素指定給變數 name。

5-5 for 迴圈與資料儲存容器

範例 5-5-3 使用 for 讀取字典

( ：ch5\5-5-3-for-dict.py)

範例	執行結果

lang={' 你好 ':'Hello',' 謝謝 ':'Thanks'}
for ch, en in lang.items():
  print(' 中文爲 ', ch, ' 英文爲 ', en)
for ch in lang.keys():
  print(ch,lang[ch])
for en in lang.values():
  print(en)
	中文爲 你好 英文爲 Hello
	中文爲 謝謝 英文爲 Thanks
	你好 Hello
	謝謝 Thanks
	Hello
	Thanks

說明

※lang 爲字典，內容爲「' 你好 ':'Hello',' 謝謝 ':'Thanks'」。

※ 使用「for ch, en in lang.items():」，使用函式 items 取出字典 lang 的「鍵」與「值」，ch 對應到「鍵」而 en 對應到「值」，接著使用函式 print 印出字典 lang 的每一個元素的「鍵」與「值」。

※ 使用「for ch in lang.keys():」，使用函式 keys 取出字典 lang 的「鍵」指定給 ch，使用 lang[ch] 取出字典的「值」，接著使用函式 print 印出字典 lang 的每一個元素的「鍵」與「值」。

※ 使用「for en in lang.values():」，使用函式 values 取出字典 lang 的「值」指定給 en，使用 en 取出字典的「值」，接著使用函式 print 印出字典 lang 的每一個元素的「值」。

5-6 生成式(comprehension)與產生器(generator)

- 生成式可以依照規則產生資料，接著將資料儲存在串列、字典與集合內。

5-6-1　串列生成式

- 若要產生一個串列有5 個數字，而5 個數字分別是「1、2、3、4 與5」，我們可以使用以下方式產生這個串列。

```
a = []
a.append(1)
a.append(2)
a.append(3)
a.append(4)
a.append(5)
```

- 或者使用for 迴圈也可以產生此串列。

```
a = []
for i in range(1,6):
    a.append(i)
```

5-6-1　串列生成式

- Python 提供另一種產生的方式為生成式(comprehension)，比較符合Python 的風格，串列生成式的語法為「[運算式 for 元素 in 可迭代的物件]」
- 使用「[ ]」表示為串列，可迭代的物件可以是tuple、串列、字串、字典與函式range等，剛才範例使用生成式改寫後如下。
  - [x for x in range(1,6)]

- 前後使用「[]」代表是串列，這行程式可以產生一個串列，串列有5 個數字，5個數字分別是「1、2、3、4 與5」。

5-6-1　串列生成式

綜合前面的所有敘述，串列生成式範例 1( ：ch5\5-6-1a-list.py)，如下表。

範例	執行結果
a = [] a.append(1) a.append(2) a.append(3) a.append(4) a.append(5) print(a) a = [] for i in range(1,6): 　a.append(i) print(a) a = [x for x in range(1,6)] print(a)	[1, 2, 3, 4, 5] [1, 2, 3, 4, 5] [1, 2, 3, 4, 5]

5-6-1　串列生成式

- 生成式也可以加上條件判斷，加上條件判斷的串列生成式格式為「[運算式 for 元素 in 可迭代的物件 if 條件判斷]」。
- 產生一個串列，該串列元素為1 到10 的所有奇數數字，使用生成式可以寫成以下程式。
  - [x for x in range(1,11) if x%2 == 1]

5-6-1　串列生成式

- 生成式也可以包含另一個生成式，前面生成式執行一次，後面接的生成式要執行完畢，形成巢狀生成式，巢狀生成式的格式為「運算式 for 元素1 in 可迭代的物件1 for 元素2 in 可迭代的物件2」。

- 以下程式會製作九九乘法表，串列每一個元素都是tuple，紀錄九九乘法表的被乘數、乘數與積的結果。
  - [(i, j, i*j) for i in range(1, 10) for j in range(1, 10)]

5-6-1　串列生成式

綜合前面的所有敘述，串列生成式範例 2 ( ：ch5\5-6-1b-list.py)

範例	執行結果
odd = [num for num in range(1,11) if (num % 2) == 1] print(odd) i = range(1, 3) j = range(1, 4) mul = [(x, y) for x in i for y in j] print(mul) for x, y in mul: 　print(x, y)	[1, 3, 5, 7, 9] [(1, 1), (1, 2), (1, 3), (2, 1), (2, 2), (2, 3)] 1 1 1 2 1 3 2 1 2 2 2 3

5-6-2　字典生成式

- 除了串列生成式外，字典也可以有生成式，要在生成式中指定出「鍵」與「值」的對應，字典生成式的語法為「{ 鍵: 值 for 元素 in 可迭代的物件}」，使用「{ 鍵:值}」表示為字典

5-6-2　字典生成式

- 要將英文字母與該字母出現的次數製作成字典，字母為字典的「鍵」，字母出現次數為字典的「值」，就可以產生一個字典生成式如下。
  - word = 'elephant'
  - {letter : word.count(letter) for letter in set(word)}

- 變數word 設定為字串「elephant」，使用 set(word) 取出字串word 的每一個字母變成一個集合
- 集合會自動去除重複的字母，而函式 word.count 會計算輸入的字母letter 在word 出現的次數，生成式產生的字典「鍵」為letter，對應到的「值」為word.count(letter)。

5-6-2 字典生成式

綜合前面敘述，字典生成式範例 ( ：ch5\5-6-2-dict.py)，如下表。

範例	執行結果
word = 'elephant' word_count = {letter : word.count(letter) for letter in set(word)} for w in word_count: print(w,word_count[w])	p 1 t 1 a 1 l 1 n 1 e 2 h 1

5-6-3　集合生成式

- 集合生成式要在生成式中指定「集合的元素」，集合生成式的語法為「{ 運算式for 元素 in 可迭代的物件}」，使用「{ 元素}」表示為集合

- 要將字串中的英文字母製作出一個集合，集合生成式如下。

```
word = 'elephant'
{letter for letter in word}
```

  - 變數word 設定為字串「elephant」，使用「letter for letter in word」取出字串word的每一個字母，外層的「{}」表示放到集合內，集合會自動去除重複的字母。

5-6-3 集合生成式

綜合前面敘述，集合生成式範例 ( ：ch5\ 5-6-3-set.py)，如下表。

範例	執行結果
word = 'elephant' letters = {letter for letter in word} for i in letters: print(i)	l n e t h p a

5-6-4　產生器(generator)

- Python 中沒有tuple 的生成式，可以使用「(運算式 for 元素 in 可迭代的物件)」製作產生器(generator)，這不是tuple 生成式，使用「( )」表示為產生器。
- 製作1 到10 的奇數產生器
  - 產生器為「(num for num in range(1,10) if (num % 2) == 1) 」
  - 產生器只能執行一次，執行第二次不會產生任何值。

5-6-4　產生器(generator)

綜合前面敘述，產生器範例 ( ：ch5\ 5-6-4-generator.py)，如下表。

範例	執行結果
odd = (num for num in range(1,10) if (num % 2) == 1) print(odd) for num in odd: 　print(num) for num in odd: 　print(num)	第二次的「for num in odd: 　　print(num)」不會輸出任何資料，因爲產生器只能使用一次。 <generator object <genexpr> at 0x00AFB9E8> 1 3 5 7 9

範例5-7-1 費氏數列 (ch5\5-7-1- 費氏數列.py)

- 費氏數列是將第1 項與第2 項相加等於第3 項，第2 項與第3 項相加等於第4 項，依此類推，初始化費氏數列的第1 項為1 且第2 項為1，計算出前n 項的費氏數列，n由使用者輸入。
- 解題想法
  - 使用「a, b = b, a+b」定義費氏數列的關係，第k+2 項等於第k 項加上第k+1 項，使用for 迴圈產生費氏數列。

範例5-7-1 費氏數列 (ch5\5-7-1- 費氏數列.py)



行數	程式碼	執行結果
1 2 3 4 5 6 7	num = int(input(' 輸入求第幾項費氏數列？ ')) a = 1 b = 1 print(1,a) for i in range(2, num+1): a, b = b, a+b print(i, a)	輸入費氏數列項數，例如：「10」，在螢幕顯示費氏數列前 10 項。 輸入求第幾項費氏數列？ 10 1 1 2 1 3 2 4 3 5 5 6 8 7 13 8 21 9 34 10 55

範例5-7-1 費氏數列 (ch5\5-7-1- 費氏數列.py)

- 程式解說
  - 第1 行：於螢幕顯示「輸入求第幾項費氏數列？」，允許使用者輸入項數，並經由函式int轉換成整數物件，變數num 參考到此整數物件。
  - 第2 行：變數 a 初始化為 1。
  - 第3 行：變數 b 初始化為 1。

- 第4 行：使用函式print 顯示數字「1」與變數a。
- 第5 到7 行：使用for 迴圈，數字2 到num 依序指定給變數i，控制迴圈執行次數與變數i 的值，使用「a, b = b, a+b」定義費氏數列的關係
- 第k+2 項等於第k 項加上第k+1 項(第6 行)
- 使用函式print 顯示變數i 與變數a(第7 行)。

範例5-7-2 產生字與字的出現次數的字典

- (ch5\5-7-2- 字與字的出現次數的字典.py)
- 使用「李白 將進酒」的詩，找出每一個字與字的出現次數，使用字當成「鍵」，出現次數當成「值」，且出現次數大於2 次的字製作成字典，接著找出字典出現次數等於3 次的字製作成另一個字典。

- 解題想法
  - 使用字典生成式製作字典，使用函式count 計算字的次數，函式set 製作字的集合避免重複計算相同的字。

範例5-7-2 產生字與字的出現次數的字典

程式碼

行數	程式碼	執行結果
1	s = '君不見黃河之水天上來，奔流到海不復迴。\	{'酒': 3, '千': 3, '生': 3, '來': 3, '君': 6, '爲': 3, '不': 5, '須': 3, '金': 3} {'酒': 3, '千': 3, '生': 3, '來': 3, '須': 3, '爲': 3, '金': 3}
2	君不見高堂明鏡悲白髮，朝如青絲暮成雪。\	
3	人生得意須盡歡，莫使金樽空對月。\	
4	天生我材必有用，千金散盡還復來。\	
5	烹羊宰牛且爲樂，會須一飲三百杯。\	
6	岑夫子，丹丘生。將進酒，君莫停。\	
7	與君歌一曲，請君爲我側耳聽。\	
8	鐘鼓饌玉不足貴，但願長醉不願醒。\	
9	古來聖賢皆寂寞，惟有飲者留其名。\	
10	陳王昔時宴平樂，斗酒十千恣讙謔。\	
11	主人何爲言少錢，徑須沽取對君酌。五花馬，千金裘。\	
12	呼兒將出換美酒，與爾同銷萬古愁。'　# 將進酒　李白	
13	dic = {w:s.count(w) for w in set(s) if w != '，' and w != '。' and s.count(w) > 2}	
14	print(dic)	
15	dic2 = {k:v for k,v in dic.items() if v ==3}	
16	print(dic2)	

範例5-7-2 產生字與字的出現次數的字典

- 第1 到12 行：將「李白 將進酒」的詩，指定給變數s。

- 第13 行：使用for 迴圈，與函式set(s) 取出變數s 中的每個文字轉換成集合，集合會自動去除重複的文字，集合set(s) 的每個元素到變數w，接著函式s.count(w) 會計算輸入的文字w 在s 出現的次數，若文字w 不等於「，」且文字w 不等於「。」
- 且文字w在s 出現的次數「s.count(w)」大於2，才加入字典，生成式產生的字典「鍵」為文字w，對應到的「值」為文字w 在s 出現的次數「s.count(w)」，變數dic 參考到此字典。

- 第14 行：使用函式print 顯示變數dic到螢幕上。
- 第15行：使用for迴圈與dic.items()取出字典dic中的每個字典元素，字典dic的「鍵」指定給變數k，「值」指定給變數v，若v 等於3，則將k 與v 的對應加入字典，生成式產生的字典「鍵」為k，對應到的「值」為v，指定給變數dic2。
- 第16 行：使用函式print 顯示變數dic2到螢幕上。

範例5-7-3 英文加密 (ch5\5-7-3- 英文加密.py)

- 使用「a 轉換成b，b 轉換為a」、「c 轉換成d，d 轉換為c」、「e 轉換成f，f 轉換為e」…「y 轉換成z，轉換為y」，進行字串的加密與解密，輸入資料一律轉成小寫。

解題想法

- 使用字典生成式製作轉換字典，根據a 到對應的ASCII 為97 到122，使用函式ord將小寫英文字元轉換成ASCII 碼，函式chr 將ASCII 碼轉換成英文字元。

- ASCII 碼為奇數的小寫英文字元，轉換成ASCII 碼為偶數的小寫英文字元，而ASCII 碼為偶數的小寫英文字元，轉換成ASCII 碼為奇數的小寫英文字元
- 再使用字典的函式update 合併兩個字典，有了轉換的字典後，就可以使用字典查詢製作加密與解密程式。

範例5-7-3 英文加密 (ch5\5-7-3- 英文加密.py)

行數	程式碼
1	sec1 = {chr(ord('a')+i): chr(ord('a')+i-1) for i in range(26) if i % 2}
2	sec2 = {chr(ord('a')+i): chr(ord('a')+i+1) for i in range(26) if i % 2 == 0}
3	sec1.update(sec2)
4	print(sec1)
5	s = 'an apple a day keeps the doctor away'
6	print(s)
7	ss = ''
8	for c in s:
9	if c != ' ':
10	ss += sec1[c]
11	else:
12	ss += ' '
13	print(ss)
14	us = ''
15	for c in ss:
16	if c != ' ':
17	us += sec1[c]
18	else:
19	us += ' '
20	print(us)



範例5-7-3 英文加密 (ch5\5-7-3- 英文加密.py)

執行結果

```
{'b': 'a', 's': 't', 'd': 'c', 'l': 'k', 'g': 'h', 'r': 'q', 'u': 'v', 'x': 'w', 'q': 'r', 'z': 'y', 'a': 'b', 'y': 'z', 'c': 'd',
'j': 'i', 'm': 'n', 'f': 'e', 'p': 'o', 'i': 'j', 'w': 'x', 'k': 'l', 'e': 'f', 't': 's', 'n': 'm', 'v': 'u', 'h': 'g', 'o': 'p'}
an apple a day keeps the doctor away
bm bookf b cbz lffot sgf cpdspq bxbz
an apple a day keeps the doctor away
```

程式解說

- 第1行：使用「for i in range(26) if i % 2」產生數字1、3、5、…、25依序指定給變數i，函式ord 將小寫英文字元轉換成ASCII 碼，因為字元a 的ASCII 碼為97，所以ord('a')為97。

程式解說

- 若變數i 等於1，則「chr(ord('a')+i)」產生字元「b」，「chr(ord('a')+i-1)」產生字元「a」，產生字元「b」對應字元「a」的字典元素

程式解說

- 若變數i 等於3，則「chr(ord('a')+i)」產生字元「d」，「chr(ord('a')+i-1)」產生字元「c」，產生字元「d」對應字元「c」的字典元素

程式解說

- 依此類推，直到變數i 等於25，則「chr(ord('a')+i)」產生字元「z」，「chr(ord('a')+i-1)」產生字元「y」，產生字元「z」對應字元「y」的字典元素，將字典指定給變數sec1。

程式解說

- ASCII 碼為偶數的小寫英文字元，轉換成 ASCII 碼為奇數的小寫英文字元。

程式解說

- 第2 行：使用「for i in range(26) if i % 2 == 0」產生數字0、2、4、…、24 依序指定給變數i，函式ord 將小寫英文字元轉換成ASCII 碼，因為字元a 的ASCII 碼為97，所以ord('a') 為97

程式解說

- 若變數i 等於0，則「chr(ord('a')+i)」產生字元「a」，「chr(ord('a')+i+1)」產生字元「b」，產生字元「a」對應字元「b」的字典元素

程式解說

- 若變數i 等於2，則「chr(ord('a')+i)」產生字元「c」，「chr(ord('a')+i+1)」產生字元「d」，產生字元「c」對應字元「d」的字典元素
- 依此類推，直到變數i 等於24，則「chr(ord('a')+i)」產生字元「y」，「chr(ord('a')+i+1)」產生字元「z」，產生字元「y」對應字元「z」的字典元素，將字典指定給變數sec2

程式解說

- ASCII 碼為奇數的小寫英文字元，轉換成 ASCII 碼為偶數的小寫英文字元。

程式解說

- 第3行：使用字典sec1的函式update，以字典sec1為基礎，將字典sec2的資料加入，若有相同的「鍵」，字典sec2 的「值」取代字典sec1 的「值」。

程式解說

- 第4 行：使用函式print 顯示字典sec1。
- 第5 行：變數 s參考到字串「an apple a day keeps the doctor away」。
- 第6 行：使用函式print 顯示變數s。